Nama : Flavian Alberta Widyadhana
Kelas : TI-1G
No. Presensi : 8

Percobaan 1

1. Komponen perulangan FOR pada program SiakadForJava10 terdiri dari tiga bagian, yaitu:

a. Inisialisasi: int i = 1
Digunakan untuk menentukan nilai awal variabel i, yaitu 1. Perulangan dimulai dari mahasiswa ke-1.

b. Kondisi: i <= 10
Digunakan untuk menentukan apakah perulangan akan dilanjutkan. Selama nilai i kurang dari atau sama dengan 10, program akan terus meminta input nilai mahasiswa.

c. Increment: i++
Digunakan untuk menambah nilai i sebanyak 1 setiap kali perulangan selesai dijalankan. Perulangan berhenti setelah nilai seluruh 10 mahasiswa dimasukkan.

2. Mengapa variabel tertinggi diinisialisasi 0 dan terendah diinisialisasi 100? Apa yang terjadi jika variabel tertinggi diinisialisasi 100 dan terendah diinisialisasi 0?
Variabel tertinggi diinisialisasi dengan nilai 0 karena digunakan untuk mencari nilai tertinggi mahasiswa. Sementara itu, variabel terendah diinisialisasi dengan nilai 100 karena digunakan untuk mencari nilai terendah mahasiswa.

Inisialisasi tersebut sesuai dengan asumsi bahwa nilai mahasiswa berada dalam rentang 0 sampai 100. Dengan begitu, nilai yang dimasukkan dapat dibandingkan dengan nilai awal untuk mendapatkan nilai tertinggi dan terendah.

Jika inisialisasinya dibalik menjadi tertinggi = 100 dan terendah = 0, hasilnya bisa salah. Nilai tertinggi kemungkinan tetap 100 karena tidak ada nilai mahasiswa yang lebih besar dari 100. Nilai terendah kemungkinan tetap 0 karena tidak ada nilai mahasiswa yang lebih kecil dari 0. Akibatnya, program tidak dapat menentukan nilai tertinggi dan terendah yang sebenarnya.

3. Potongan kode tersebut berfungsi untuk menentukan nilai tertinggi dan terendah mahasiswa.

Pada bagian pertama, program memeriksa apakah nilai mahasiswa yang baru dimasukkan lebih besar daripada variabel tertinggi. Jika kondisi benar, variabel tertinggi akan diperbarui dengan nilai mahasiswa tersebut.
Pada bagian kedua, program memeriksa apakah nilai mahasiswa yang baru dimasukkan lebih kecil daripada variabel terendah. Jika kondisi benar, variabel terendah akan diperbarui.
Proses tersebut dilakukan berulang kali untuk setiap mahasiswa. Setelah semua nilai dimasukkan, program akan mendapatkan nilai tertinggi dan terendah dari 10 mahasiswa.

4. Source Code setelah modifikasi

```java
import java.util.Scanner;Use Ctrl + I to generate code

public class SiakadForJava10 {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        double nilai;
        double tertinggi = 0;
        double terendah = 100;
        int lulus = 0;
        int tidakLulus = 0;

        for (int i = 1; i <= 10; i++) {
            System.out.print("Masukkan nilai mahasiswa ke-" + i + ": ");
            nilai = sc.nextDouble();

            if (nilai > tertinggi) {
                tertinggi = nilai;
            }

            if (nilai < terendah) {
                terendah = nilai;
            }

            if (nilai >= 60) {
                lulus++;
            } else {
                tidakLulus++;
            }
        }

        System.out.println("Nilai tertinggi: " + tertinggi);
        System.out.println("Nilai terendah: " + terendah);
        System.out.println("Jumlah mahasiswa lulus: " + lulus);
        System.out.println("Jumlah mahasiswa tidak lulus: " + tidakLulus);

        sc.close();
    }
}
```

Output:

```
Masukkan nilai mahasiswa ke-1: 98
Masukkan nilai mahasiswa ke-2: 90
Masukkan nilai mahasiswa ke-3: 93
Masukkan nilai mahasiswa ke-4: 60
Masukkan nilai mahasiswa ke-5: 50
Masukkan nilai mahasiswa ke-6: 30
Masukkan nilai mahasiswa ke-7: 90
Masukkan nilai mahasiswa ke-8: 100
Masukkan nilai mahasiswa ke-9: 37
Masukkan nilai mahasiswa ke-10: 45
Nilai tertinggi: 100.0
Nilai terendah: 30.0
Jumlah mahasiswa lulus: 6
Jumlah mahasiswa tidak lulus: 4
```

Percobaan 2

1. Pada potongan kode berikut, tentukan maksud dan kegunaan dari sintaks tersebut.
a. nilai < 0 || nilai > 100
Sintaks tersebut digunakan untuk memeriksa apakah nilai mahasiswa tidak valid. Operator || berarti OR (atau), yaitu kondisi akan bernilai benar apabila salah satu atau kedua kondisi terpenuhi.

- nilai < 0 berarti nilai kurang dari 0.
- nilai > 100 berarti nilai lebih dari 100.

Jika nilai kurang dari 0 atau lebih dari 100, program akan menampilkan pesan bahwa nilai tidak valid. Jika kondisi tersebut terpenuhi, program menjalankan continue agar input nilai diulang.

b. continue
Sintaks continue digunakan untuk melewati sisa perintah pada iterasi perulangan saat ini dan melanjutkan ke iterasi berikutnya.

Pada program SiakadWhile10, jika nilai yang dimasukkan tidak valid, perintah continue akan membuat program kembali ke awal perulangan while untuk meminta nilai mahasiswa yang sama. Variabel i tidak bertambah karena i++ berada di bawah perintah tersebut. Dengan demikian, jumlah mahasiswa yang harus diinput tetap sama.

2. Sintaks i++ dituliskan di akhir perulangan agar variabel i bertambah setelah nilai mahasiswa berhasil dimasukkan dan diproses. Dengan demikian, setiap mahasiswa mendapatkan nomor urut yang sesuai dan jumlah mahasiswa yang diproses tetap tepat.

Jika i++ dituliskan di awal perulangan, nomor urut mahasiswa dapat dimulai dari angka yang salah karena nilai i bertambah sebelum input diproses. Selain itu, jika penempatannya tidak tepat, perulangan dapat melewatkan mahasiswa atau membuat jumlah mahasiswa yang diproses tidak sesuai. Oleh karena itu, pada program ini, i++ sebaiknya diletakkan setelah nilai mahasiswa berhasil divalidasi dan diproses.

3. Perulangan while akan berjalan sebanyak 19 kali untuk memproses 19 mahasiswa, dengan asumsi seluruh nilai yang dimasukkan valid.
Namun, jika terdapat nilai yang tidak valid, perulangan akan berjalan lebih dari 19 kali karena program meminta input ulang untuk mahasiswa yang sama sampai mendapatkan nilai valid.

4. Source Code

```java
import java.util.Scanner;

public class SiakadWhile10 {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
    Scanner sc = new Scanner (System.in);
//deklarasi
    int nilai;
    int jml;
    int i = 0;

    System.out.print ("Masukkan jumlah mahasiswa: ");
    jml = sc.nextInt();

    while (i < jml){
    System.out.print("Masukkan nilai mahasiswa ke " + (i + 1) + ": ");
    nilai = sc.nextInt();

    if (nilai < 0 || nilai > 100) {
        System.out.println("Nilai tidak valid, masukkan lagi nilai yang valid");
        continue;
            }
    if (nilai > 80 && nilai <= 100) {
        System.out.println("Nilai mahasiswa ke " + (i+1) + " adalah A");
        System.out.println("Bagus, pertahankan nilainya");

            }
    else if (nilai > 73 && nilai <= 80) {
        System.out.println("Nilai mahasiswa ke " + (i+1) + " adalah B+");
    }
    else if (nilai > 65 && nilai <= 73) {
        System.out.println("Nilai mahasiswa ke " + (i+1) + " adalah B");
        }
    else if (nilai > 60 && nilai <= 65) {
        System.out.println("Nilai mahasiswa ke " + (i+1) + " adalah C+");
        }
    else if (nilai > 50 && nilai <= 60) {
        System.out.println("Nilai mahasiswa ke " + (i+1) + " adalah C");
        }
    else if (nilai > 39 && nilai <= 50) {
        System.out.println("Nilai mahasiswa ke " + (i+1) + " adalah D");
        }
    else {
        System.out.println("Nilai mahasiswa ke " + (i+1) + " adalah E");
    }
    i++;
        }
    }
}
```

Output:

```
Masukkan jumlah mahasiswa: 2
Masukkan nilai mahasiswa ke 1: 10
Nilai mahasiswa ke 1 adalah E
Masukkan nilai mahasiswa ke 2: 90
Nilai mahasiswa ke 2 adalah A
Bagus, pertahankan nilainya
```

Percobaan 3

1. Apabila nama pelanggan yang dimasukkan pertama kali adalah "batal", perulangan do-while hanya melakukan satu kali iterasi. Hal ini karena perulangan do-while menjalankan blok kode terlebih dahulu sebelum memeriksa kondisi. Ketika nama pelanggan sama dengan "batal", program menjalankan perintah break sehingga perulangan langsung dihentikan tanpa meminta jumlah kopi, teh, dan roti. Setelah itu, program menampilkan pesan "Transaksi dibatalkan." dan "Semua transaksi selesai."

2. Kondisi berhenti yang digunakan pada program tersebut adalah nama pelanggan sama dengan "batal", yang diperiksa melalui namaPelanggan.equalsIgnoreCase("batal"). Jika kondisi tersebut terpenuhi, perintah break akan menghentikan perulangan secara langsung. Selain itu, kondisi pada bagian akhir perulangan adalah while (true), yang berarti perulangan akan terus berjalan selama tidak dihentikan oleh perintah break.

3. Nilai true pada kondisi do-while berfungsi untuk membuat perulangan berjalan terus-menerus karena kondisi tersebut selalu bernilai benar. Dengan demikian, program dapat melayani transaksi pelanggan berkali-kali tanpa menentukan jumlah perulangan sejak awal. Perulangan hanya akan berhenti ketika pelanggan memasukkan nama "batal" sehingga kondisi tersebut memicu perintah break.

4. Perulangan do-while tetap berjalan meskipun tidak memiliki komponen inisialisasi dan update seperti perulangan for, karena do-while tidak memerlukan komponen tersebut secara khusus. Perulangan ini menjalankan blok kode terlebih dahulu, kemudian memeriksa kondisi di bagian akhir. Pada program tersebut, kondisi while (true) selalu bernilai benar sehingga perulangan terus berjalan. Penghentian perulangan dilakukan menggunakan perintah break ketika pelanggan memasukkan nama "batal".

Nama : Flavian Alberta Widyadhana
Kelas : TI-1G
No. Presensi : 8

Tugas 1

Source code:

```
import      public class TiketBioskop10
            extends Object
public class TiketBioskop10 {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int jumlahTiket;
        int hargaTiket = 50000;
        int totalPenjualan = 0;
        double diskon, totalBayar;

        while (true) {
            System.out.print("Masukkan jumlah tiket (0 untuk selesai): ");
            jumlahTiket = sc.nextInt();

            if (jumlahTiket == 0) {
                break;
            }

            if (jumlahTiket < 0) {
                System.out.println("Jumlah tiket tidak valid. Ulangi input.");
                continue;
            }

            diskon = 0;

            if (jumlahTiket > 10) {
                diskon = 0.15;
            } else if (jumlahTiket > 4) {
                diskon = 0.10;
            }

            totalBayar = jumlahTiket * hargaTiket;
            totalBayar -= totalBayar * diskon;
            totalPenjualan += (int) totalBayar;

            System.out.println("Jumlah tiket: " + jumlahTiket);
            System.out.println("Diskon: " + (diskon * 100) + "%");
            System.out.println("Total pembayaran: Rp " + (int) totalBayar);
        }

        System.out.println("Total penjualan hari ini: Rp " + totalPenjualan);

        sc.close();
    }
}
```

Output:

```
Masukkan jumlah tiket (0 untuk selesai): 10
Jumlah tiket: 10
Diskon: 10.0%
Total pembayaran: Rp 450000
Masukkan jumlah tiket (0 untuk selesai):
```

Penjelasan Kode:

Program TiketBioskop10 digunakan untuk menghitung total pembayaran pembelian tiket bioskop berdasarkan jumlah tiket yang dibeli. Program menggunakan Scanner untuk menerima input jumlah tiket dari pengguna, variabel hargaTiket sebesar Rp50.000 sebagai harga satu tiket, dan totalPenjualan untuk menyimpan akumulasi penjualan. Perulangan while (true) digunakan agar pengguna dapat melakukan input pembelian berulang kali. Jika pengguna memasukkan angka 0, perintah break akan menghentikan perulangan. Jika jumlah tiket kurang dari 0, program menampilkan pesan bahwa jumlah tiket tidak valid dan menjalankan continue untuk meminta input kembali tanpa memproses transaksi tersebut.

Program menentukan diskon berdasarkan jumlah tiket yang dibeli, yaitu 15% jika membeli lebih dari 10 tiket, 10% jika membeli lebih dari 4 hingga 10 tiket, dan tidak mendapat diskon jika membeli 4 tiket atau kurang. Selanjutnya, program menghitung total pembayaran dengan mengalikan jumlah tiket dengan harga tiket, kemudian mengurangi hasilnya sesuai diskon yang diperoleh. Nilai pembayaran ditambahkan ke variabel totalPenjualan untuk menghitung keseluruhan penjualan hari itu. Program menampilkan jumlah tiket, persentase diskon, dan total pembayaran setiap transaksi. Setelah pengguna memasukkan angka 0, program menampilkan total penjualan hari ini, lalu menutup Scanner menggunakan sc.close().

Tugas 2

Source Code

```java
import java.util.Scanner;

public class Parkir10 {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int jenis, durasi, total = 0;

        do {
            System.out.print("Masukkan jenis kendaraan (1 mobil, 2 motor, 0 keluar): ");
            jenis = sc.nextInt();

            if (jenis == 1 || jenis == 2) {
                System.out.print("Masukkan durasi parkir (jam): ");
                durasi = sc.nextInt();

                if (jenis == 1) {
                    total += durasi * 3000;
                } else {
                    total += durasi * 2000;
                }
            } else if (jenis != 0) {
                System.out.println("Jenis kendaraan tidak valid.");
            }

        } while (jenis != 0);

        System.out.println("Total pembayaran parkir: Rp " + total);

        sc.close();
    }
}
```

Output:

```
Masukkan jenis kendaraan (1 mobil, 2 motor, 0 keluar): 2
Masukkan durasi parkir (jam): 5
Masukkan jenis kendaraan (1 mobil, 2 motor, 0 keluar): 1
Masukkan durasi parkir (jam): 10
Masukkan jenis kendaraan (1 mobil, 2 motor, 0 keluar): 0
Total pembayaran parkir: Rp 40000
```

Penjelasan Kode:

Program Parkir10 digunakan untuk menghitung total biaya parkir berdasarkan jenis kendaraan dan durasi parkir. Program menggunakan Scanner untuk menerima input dari pengguna, variabel jenis untuk menyimpan pilihan kendaraan, variabel durasi untuk menyimpan lama parkir dalam jam, dan variabel total untuk menyimpan akumulasi biaya parkir. Perulangan do-while digunakan agar pengguna dapat memasukkan beberapa kendaraan secara berulang. Jika pengguna memasukkan angka 1, kendaraan dianggap sebagai mobil, sedangkan angka 2 menunjukkan motor. Program kemudian meminta durasi parkir kendaraan tersebut. Jika pengguna memasukkan angka selain 0, 1, atau 2, program akan menampilkan pesan bahwa jenis kendaraan tidak valid.

Program menghitung biaya parkir mobil sebesar Rp3.000 per jam dan motor sebesar Rp2.000 per jam. Jika kendaraan merupakan mobil, biaya dihitung menggunakan durasi * 3000, sedangkan untuk motor menggunakan durasi * 2000. Hasil perhitungan ditambahkan ke variabel total menggunakan operator += agar biaya seluruh kendaraan terakumulasi. Perulangan akan terus berjalan selama nilai jenis tidak sama dengan 0. Jika pengguna memasukkan angka 0, perulangan berhenti dan program menampilkan total pembayaran parkir seluruh kendaraan. Terakhir, sc.close() digunakan untuk menutup objek Scanner setelah program selesai.